

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY....3
CLIENT....3
INSTRUCTIONS....3
DEVELOPER....4
1. ALGORITHM CIPHER....4
2. CERTIFICATE GENERATION....4
3. DEPLOY CIPHER....4
4. SECURE COMMUNICATIONS....4
5. SECONDARY TESTING....4
6. FUNCTIONAL TESTING....4
7. SUMMARY....4
8. INDUSTRY STANDARD BEST PRACTICES....4

Document Revision History

Version	Date	Author	Comments
1.0	[Date]	[Your Name]	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Dayron Rodriguez

1. Algorithm Cipher

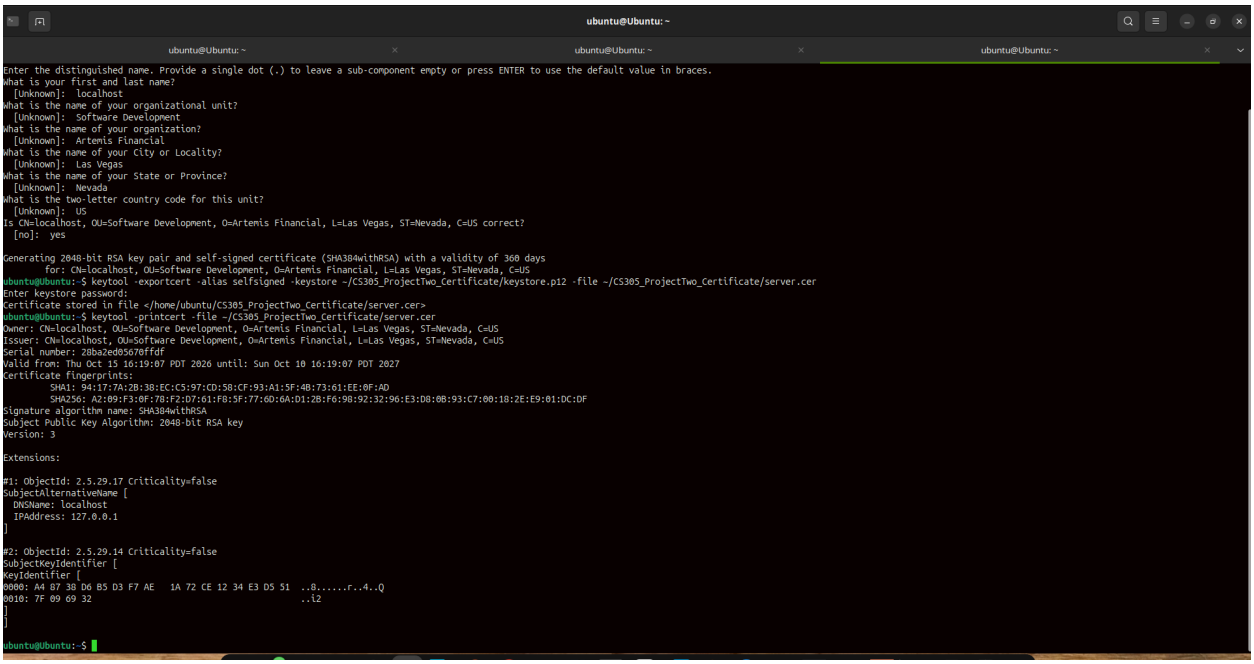
I selected SHA-256 for Artemis Financial's checksum verification. SHA-256 belongs to the SHA-2 family and produces a 256-bit hash, displayed as 64 hexadecimal characters. Comparing this checksum with a trusted original checksum helps detect changes to data. SHA-256 is a one-way hash function; it does not encrypt information or provide confidentiality (National Institute of Standards and Technology [NIST], 2015).
For encryption, I recommend AES-256 within a properly configured TLS connection. AES-256 uses a 256-bit symmetric key, meaning the same secret key encrypts and decrypts data. My project also uses a certificate with a 2048-bit RSA asymmetric key pair, consisting of a public key and a private key. The certificate supports server authentication, while TLS protects data during transmission. AES and SHA-256 perform different jobs: encryption protects confidentiality, and hashing supports integrity verification.

Cryptographically secure random numbers are necessary when generating keys and establishing secure connections. However, the SHA-256 checksum must remain deterministic: the same input always produces the same hash. Therefore, my checksum implementation does not add random data.

Cryptographic standards have evolved as older algorithms became vulnerable. AES was standardized in 2001, and its specification received an editorial update in 2023. NIST recommends transitioning away from SHA-1 toward SHA-2 or SHA-3. Selecting established algorithms and reviewing them regularly helps Artemis Financial maintain its security as threats change (NIST, 2022, 2023).

2. Certificate Generation

Insert a screenshot below of the CER file.



3. Deploy Cipher

Insert a screenshot below of the checksum verification.



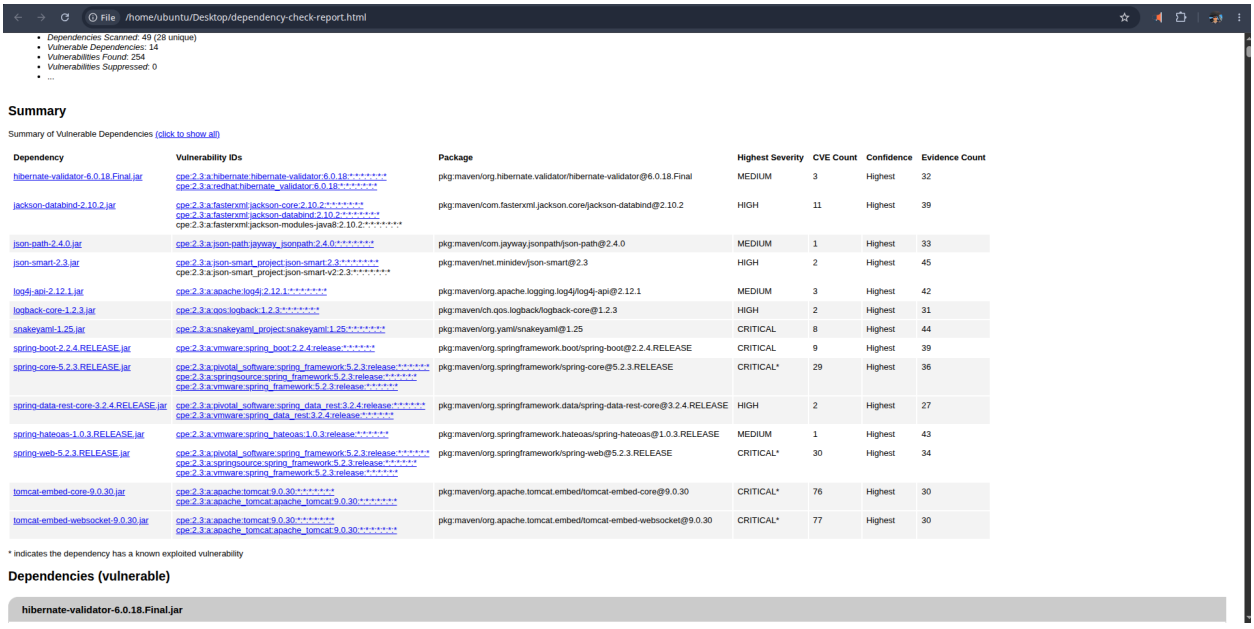
4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.



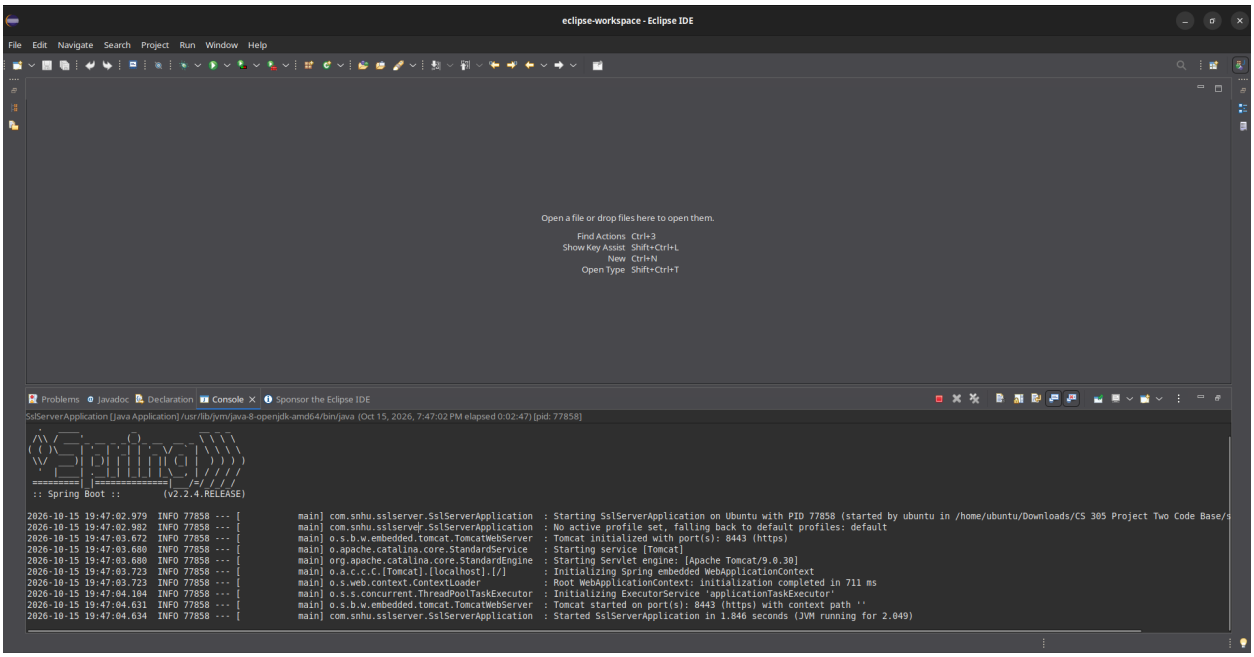
5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.



6.Functional Testing

Insert a screenshot below of the refactored code executed without errors.



6. Summary

I refactored Artemis Financial's application by adding a SHA-256 checksum endpoint and enabling HTTPS on port 8443. The endpoint displays my name, a unique data string, and its checksum. I also generated a self-signed RSA certificate, exported it as a CER file, and configured the application to load its PKCS12 keystore. These changes add integrity checking and encrypted communication.

Following the vulnerability assessment process flow diagram, I reviewed input validation, APIs, cryptography, client/server communication, error handling, code quality, and encapsulation. The implementation uses a fixed input string, standard Java cryptographic APIs, explicit UTF-8 encoding, and a private hexadecimal conversion method. The keystore password is supplied through an environment variable. I verified successful application startup and checked the checksum displayed in the browser.

I also ran OWASP Dependency-Check to review third-party library risks. The scan completed successfully but reported existing dependency vulnerabilities that still require investigation and remediation. The browser also displayed a trust

warning for the self-signed certificate. These results show that checksum generation and local HTTPS work, while dependency updates and certificate trust remain necessary before production deployment.

7. Industry Standard Best Practices

I applied secure coding practices by using Java's established MessageDigest API for SHA-256 instead of creating a custom hashing algorithm. I used UTF-8 encoding for consistent results and verified the generated checksum. HTTPS adds protection for data in transit, while an environment variable keeps the keystore password out of the application properties file.
I also manually reviewed the added code and ran OWASP Dependency-Check. The remaining dependency findings require investigation, updates, and retesting. The self-signed certificate supports local testing, but production deployment requires a certificate trusted by the intended clients.

These practices help Artemis Financial protect sensitive financial information, reduce security risks, and maintain customer trust. Regular code reviews, dependency updates, and testing help preserve these protections as the application changes.

References:

National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (FIPS PUB 180-4). https://doi.org/10.6028/NIST.FIPS.180-4

National Institute of Standards and Technology. (2022, December 15). *NIST transitioning away from SHA-1 for all applications*. https://www.nist.gov/news-events/news/2022/12/nist-transitioning-away-sha-1-all-applications

National Institute of Standards and Technology. (2023). *Advanced encryption standard (AES)* (FIPS PUB 197). https://doi.org/10.6028/NIST.FIPS.197-upd1

Oracle. (n.d.). *MessageDigest*. Java Platform, Standard Edition 8 API specification. https://docs.oracle.com/javase/8/docs/api/java/security/MessageDigest.html

OWASP Foundation. (n.d.). *OWASP Dependency-Check*. https://dependency-check.github.io/DependencyCheck/

Southern New Hampshire University. (n.d.). *CS 305 vulnerability assessment process flow diagram* [Course diagram]. CS 305 Software Security.